Quelques Problèmes

FIP --- 2026/2027

1 Modèle de Lotka Volterra

On considère l'équation diférentielle suivante :

$$\frac{\mathrm{d}L}{\mathrm{d}t} = 2L - 0.2LR \tag{1}$$

$$\frac{\mathrm{d}R}{\mathrm{d}t} = -30R + 0.15LR \tag{2}$$

Il s'agit d'un modèle qui décrit une population de renards R et une population de lapins L qui se partagent le même territoire.

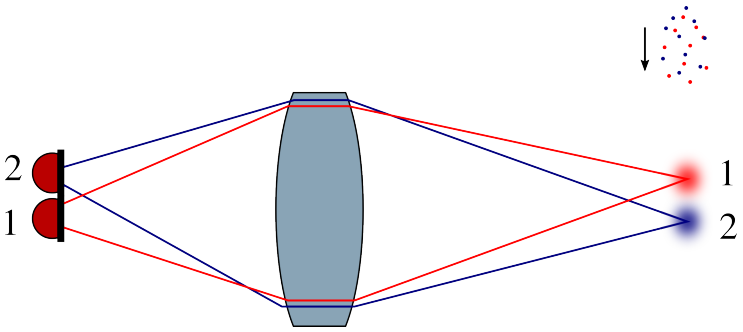
Le coefficient 2 est le taux de croissance des lapins en l'absence de renards, d'où le signe positif, 30 est le taux de décroissance des renards en l'absence de lapins, d'où le signe négatif, 0.2 est un coefficient exprimant que les rencontres tournent mal pour les lapins, d'où le -, 0.15 est un coefficient exprimant que les rencontres sont bénéfiques aux renards, d'où le +.

1. Résoudre cette équation pour $L_0 = 150$ et $R_0 = 5$ et tracer l'évolution jusque $t = 10$.
2. Avec les même paramètres, tracer la valeur absolue de la transformée de Fourier numérique du nombre de lapin (en prenant t=100) et donner approximativement la fréquence fondamentale du signal.

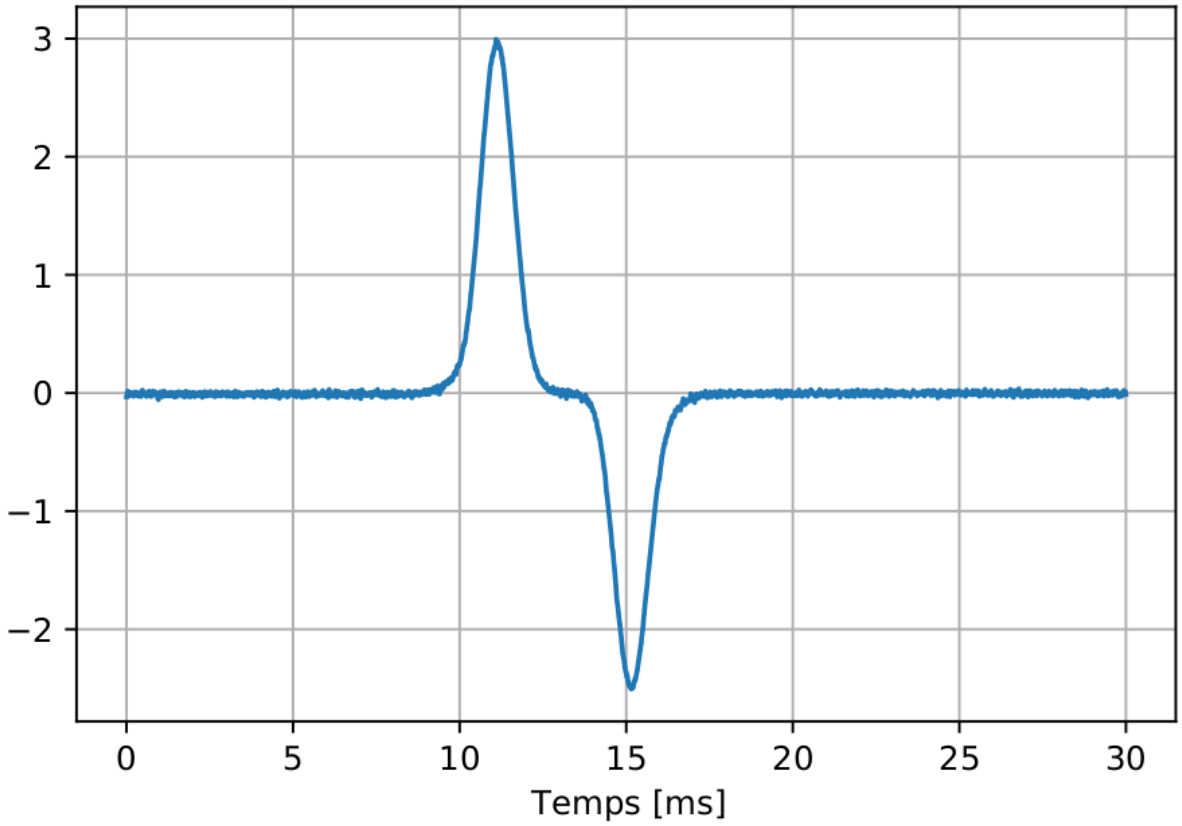
```
from scipy.integrate import solve_ivp
from numpy.fft import rfft, rfftfreq
import numpy as np
```

2 Analyse des signaux de temps de vol d'atomes

L'objectif de cet exercice est d'analyser les signaux d'un interféromètre atomique. A l'issu de l'interféromètre, les atomes sont dans deux états interne différents. Chaque atome est résonnant avec un laser spécifique (représenté en bleu et rouge). Pour pouvoir mesurer le nombre d'atome, on utilise deux nappes horizontales avec deux lasers. Lorsque les atomes tombent à travers la nappe, il vont diffuser les photons. Chaque nappe est imagée sur une photodiode différent. Ce sont d'abord les atomes labelisés 1 qui sont détectés puis ceux labelisés 2.



Le fichier `fichier_photodiode.txt` contient le signal de chaque photodiode. Le taux d'échantillonage est de 250 000 échantillons par seconde. Il y a beaucoup de bruit en commun sur chaque photodiode. Pour analyser les résultats, nous faisons la différence des signaux entre chaque photodiode. Voici le résultat:



1. Lire le fichier (on pourra utiliser np.loadtxt) et tracer la courbe comme ci dessus

L'objectif est de calculer le nombre d'atomes. Pour cela, on va utiliser deux techniques : la première consiste à intégrer l'aire sous la courbe et la seconde à ajuster par deux gaussiennes. On note N_1 le nombre d'atomes dans la première bosse et N_2 dans la seconde. Il s'agit du nombre d'atomes à un facteur prêt qui dépend de l'efficacité de la détection et du gain de la photodiode. Ce facteur s'éliminera lorsque l'on calculera la proportion d'atome dans l'état 1.

Il y a un léger offset sur les courbes (au début, lorsqu'il n'y a pas d'atomes, le signal est non nul). On calculera cet offset en prenant la moyenne entre 0 et 5 ms puis on l'enlèvera. Le nombre d'atomes N_1 sera obtenu en intégrant entre 9 et 13 ms et le nombre d'atome N_2 entre 13 et 17 ms.

2. Calculer sur ce signal N_1 et N_2

Une Gaussienne est donnée par la fonction :

$$\text{Gauss}(t) = \frac{a}{\sqrt{2\pi\sigma^2}} e^{-(t-t_0)^2/(2\sigma^2)}$$

Cette fonction est telle que l'intégrale vaut a.

3. Extraire les points entre 9 et 13 ms et afficher juste ces points (ne pas faire un xlim, mais faire deux tableaux contenant uniquement les points entre 9 et 13 ms)
4. Effectuer un fit par une gaussienne de ces points entre 9 et 13 ms et donner les paramètres optimaux.
5. Effectuer un fit de l'ensemble des points (figure ci-dessus) par une fonction qui est la somme d'un offset et de la différence de deux gaussiennes.

Les données sont en fait extraites d'un fichier contenant un grand nombre de mesure. Il y a en tout trois fichiers : `parametres.txt`, `photodiode_0.npy`, `photodiode_1.npy`. Le premier se lit avec un `np.loadtxt`, les deux suivants avec un `np.load`. Le fichier `parametres.txt` contient trois colonnes : un timestamp (donné par la

fonction time du module time()), le type de spectre et la fréquence utilisée. En effet, lors de la mesure, nous effectuons 4 types de spectres correspondant à des configurations différentes.

6. Écrire une fonction qui prend le nom des trois fichiers comme argument et un type de spectre (un entier 0, 1, 2, ou 3) et renvoie un dictionnaire avec comme clé : timestamp, frequence, tof. Cette fonction ne renverra que les lignes pour lesquels le type de spectre correspond à l'argument de la fonction. Si le spectre n'existe pas, elle renverra une erreur.
7. En utilisant la méthode `fromtimestamp` de l'objet `date` disponible dans la librairie `datetime`, donner le jour où cette mesure à été faite.
8. Calculer les nombres N_1 et N_2 par integration directement et sans faire de boucle `for` sur les temps de vol (pour valider cette question, le mot clé `for` ne doit pas être utilisé).
9. Tracer la proportion $N_1/(N_1 + N_2)$ en fonction de la fréquence.
10. Ajuster cette courbe par une sinusoïde et trouver la fréquence centrale. Donner son incertitude.
11. Mêmes questions que 8, 9 et 10, mais en faisant un fit au lieu d'une intégration (on pourra utiliser une boucle `for` ici !)

```
import numpy as np
import matplotlib.pyplot as plt

from scipy.optimize import curve_fit
```

3 Modèle d'Ising

Un système magnétique peut-être décrit de façon simple avec l'Hamiltonien suivant (modèle d'Heisenberg):

$$H = -\frac{4}{h^2}\sum_{i<j} J_{ij}\vec{s}_i \cdot \vec{s}_j - \gamma\vec{B}\cdot\sum_i \vec{s}_i \tag{3}$$

Dans cet exercice, nous allons nous intéresser à un modèle encore plus simple: celui d'un ensemble de spin 1/2 et ne considérer que l'intéraction entre les spins voisins les plus proches. En écrivant $\vec{s}_i = \frac{\hbar}{2}S_i$ ($S_i = \pm 1$), on obtient le modèle d'Ising décrit par l'Hamiltonien:

$$H = -J\sum_{\langle i,j\rangle} S_i S_j - \mu B\sum_i S_i \tag{4}$$

où la notation $\langle i, j\rangle$ denote la somme sur les paires des plus proches voisins.

Nous allons dans cet exercice simuler ce système à l'aide d'un algorithme appelé "Metrolopolis". Celui-ci est détaillé sur la page wikipedia dédié au model d'Ising. https://en.wikipedia.org/wiki/Ising_model . Nous prendrons un champ magnétique nul.

L'état du système sera représenté par un tableau de taille $N \times N$. Nous utiliserons des conditions périodiques (i.e. le spin à la position $(N-1, j)$ est couplé au spin à la position $(0, j)$, etc.

1. Écrire une fonction `energy(state)` qui retourne l'énergie totale du système magnétique dans l'état 'state'.
2. Écrire une fonction `delta_energy(state, i, j)` qui retourne la difference d'énergie induite par le flipping du spin de coordonnees `(i, j)`. Nous rappellons que seule l'intéraction entre plus proches voisins est considérée.
3. Écrire une fonction `metropolis(state, beta, M)` qui retourne l'état du système magnétique après avoir implémenté M fois l'algorithme de Metrolopolis. Une réalisation de l'algorithme correspond à choisir un point de manière aléatoire, à calculer la différence d'énergie induite par le flip de celui-ci. Si cette énergie est négative, alors on considère que le spin flip. Si la différence d'énergie est positive, on autorise le spin flip selon la probabilité: $e^{-\beta\Delta E}$.

4. Afficher l'image pour $\beta = 1$, après 1000, 1 000 000, 100 000 000 itérations. Vous pouvez aussi faire une animation. On prendra comme état initial, 300x300 spins de valeur aléatoire ± 1. Il faudra utiliser numba.

5. Nous allons regarder l'énergie moyenne du système. Modifier votre fonction metropolis, de façon à pouvoir enregistrer l'énergie moyenne du système pendant l'évolution. Pour éviter des problèmes de mémoires, nous vous conseillons de l'enregistrer uniquement tous les N^2 réalisations.

   Tracer l'énergie du système au cours du temps. Observer le temps de convergence du système (on prendra $\beta = 1$, $\beta = 0.2$ et $\beta = 0.1$). En plus de la convergence, qu'observez-vous pour $\beta = 0.2$?

6. Tracer l'énergie moyenne en fonction de β. Vous pourrez aussi accélérer votre calcul en l'évaluant sur plusieurs noyaux de votre ordinateur, à l'aide du décorateur: `@jit(parallel=True)` et de la boucle `for beta in numba.prange(xxx):`.

7. De même, tracer les fluctuations d'énergie en fonction β

8. Comme vous l'avez vu en cours de Physique Statistique, il existe une solution analytique au modèle d'Ising en deux dimensions:

$$Z = \left(2\cosh\frac{2J}{k_BT}e^I\right)^N, \quad I = \frac{1}{2\pi}\int_0^\pi \mathrm{d}\phi \log\frac{1+\sqrt{1-x^2\sin^2\theta}}{2}, \quad x = \frac{2\sinh\left(\frac{2J}{k_BT}\right)}{\cosh^2(\frac{2J}{k_BT})}$$

Comment calculer l'énergie moyenne du système à partir de la fonction de partition Z ? Evaluez numériquement cette expression et comparez à votre simulation metrolopolis.

Si vous avez le temps, même question pour les fluctuations d'énergie

9. La transition de phase a lieu pour T qui vérifie: \begin{equation} \sinh\left(\frac{2J}{kT}\right)= 1 \end{equation} Résolvez cette equation numériquement (on pourra utiliser la fonction root de scipy) et comparez aux résultats de la question précédente.

On observe des fluctuations importantes pour la courbe $\beta = 0.2$

L'énergie moyenne est donnée par :

$$\langle E\rangle = -\frac{\partial \ln Z}{\partial \beta} \tag{5}$$

Les fluctuations d'énergie par la dérivée seconde.